TFRA-YOLO: Textile Feature Refinement and Reassembly YOLO for Closed-Loop Fabric Defect Inspection on Circular Knitting Machines

Kangzhao Tan, Jiarui Zhang, Junjie Yu, Heng Wu, and Chunhua He

***Abstract*—Large circular knitting machines underpin high-throughput textile production, particularly for plain-knit fabric, one of the most extensively manufactured fabric types. Conventional inspection is commonly performed after production; therefore, a defect that emerges early can remain undiscovered while the machine continues knitting, causing a substantial length of defective fabric, material waste, and rework. This paper presents an in-process, closed-loop edge-cloud inspection workflow and TFRA-YOLO, a lightweight detector designed for this production setting. A proximity-switch rotation signal activates edge-side inspection; when a defect is detected, the edge device sends a local stop command and presents the candidate defect to an operator for secondary confirmation. Device states, defect records, and local confirmation outcomes are synchronized to a cloud digital-twin platform for real-time status visualization, defect statistics, process-improvement analysis, and authorized remote supervisory control. TFRA-YOLO starts from YOLOv8n and introduces a warp–weft texture encoding (WWTE) block that jointly models horizontal, vertical, and local texture responses, together with a local texture-guided feature reassembly (LTFR) block that replaces the two fixed top-down upsampling operations with local feature reassembly. On a real production-line dataset of 403 images covering horizontal defects, vertical defects, and holes, the retained test split contains 81 images and 91 instances. TFRA-YOLO obtains 66.3% AP and 96.9% AP50, improving YOLOv8n by 7.0 and 2.8 percentage points, respectively, with 3.39 M parameters and 8.9 G FLOPs at 640×640 input. These results support the usefulness of orthogonal texture representation and local pyramid reconstruction for lightweight, in-process plain-knit fabric inspection. The manuscript also states the deployment measurements still required before a production-performance claim is made.**

***Index Terms*—Fabric defect detection, circular knitting machine, edge intelligence, closed-loop inspection, feature pyramid, lightweight object detection.**

I. Introduction

CIRCULAR knitting machines produce fabric continuously, so an overlooked defect can propagate before an operator can intervene. A practical inspection system must therefore address more than offline recognition accuracy: it must synchronize sensing with machine operation, make a local decision without waiting for a cloud round trip, preserve evidence for local human review, and synchronize confirmed cases and device states to a cloud digital twin for status visualization, defect statistics, and process improvement. Earlier vision systems have demonstrated on-machine inspection for circular knitting processes [1], while recent work continues to identify rapid motion, illumination variation, and scarce abnormal samples as central challenges [2]. Broader reviews trace the evolution of fabric inspection from handcrafted texture analysis to learning-based methods and emphasize the simultaneous demands of localization accuracy, limited defect labels, and production-line speed [3]–[6]. CNN-based localization [7], weakly supervised detection [8], and recent lightweight detectors for circular weft fabric and edge devices [9], [10] each address parts of this problem. However, many learning-based defect-detection studies still evaluate an image classifier or detector in isolation from the production response loop.

The target images in this work exhibit a second challenge. The fabric surface is dominated by regular knitted texture, whereas a horizontal defect, a vertical defect, or a hole can be thin, low contrast, and highly anisotropic. One-stage detectors, from the original YOLO formulation [11] through anchor-free and trainable-efficiency variants [12], [13], provide a suitable starting point when the detector must remain compact. A compact detector must nevertheless retain local texture detail at the high-resolution pyramid level while also exploiting context to distinguish a true defect from benign texture changes. General multi-scale detectors use feature pyramids [14] and bottom-up aggregation [15]; content-aware feature reassembly [16] further shows that interpolation need not use a spatially fixed kernel. At the same time, asymmetric convolutional paths can strengthen directional structure [17]. These ideas are relevant to knitted fabric, but must be adapted conservatively so that the edge model remains small.

At the system level, edge computing reduces dependence on remote-cloud communication for latency-sensitive decisions [18], while digital-twin studies distinguish data-connected virtual representations from the local physical process they supervise [19], [20]. Accordingly, the validation plan in this work combines detection accuracy, model complexity, qualitative inspection, and embedded deployment measurements rather than relying on an accuracy-only presentation.

This paper makes the following contributions.

- It designs and implements a production-oriented, human-in-the-loop inspection system for a circular knitting machine, integrating a Hall sensor, a 1.3-megapixel industrial global-shutter camera, a 30-level adjustable supplementary light, a 4G wireless communication module, an RK3566 edge terminal, and an IoT digital-twin platform. The rotation-derived signal gates edge inspection, a local defect decision requests a stop, an on-site operator confirms the case, and device states plus detection-confirmation records are synchronized for real-

K. Tan, J. Zhang, J. Yu, and C. He are with the School of Computer Science, Guangdong University of Technology, Guangzhou 510006, China. H. Wu is with the School of Automation, Guangdong University of Technology, Guangzhou 510006, China. Corresponding author: C. He (e-mail: hechunhua@pku.edu.cn). E-mails: K. Tan (tankangzhao@mails.gdut.edu.cn), J. Zhang (zhangjiarui1@mail2.gdut.edu.cn), J. Yu (yujunjie2@mails.gdut.edu.cn), H. Wu (heng.wu@foxmail.com), and C. He (hechunhua@pku.edu.cn).

time visualization, defect statistics, process-improvement analysis, and authorized remote supervisory control.
- It proposes TFRA-YOLO, a YOLOv8n-based detector with a warp–weft texture encoding (WWTE) block and a local texture-guided feature reassembly (LTFR) block. WWTE preserves horizontal, vertical, and local texture cues; LTFR performs content-adaptive rather than fixed-neighbor top-down reconstruction.
- It reports a transparent test-set comparison against lightweight YOLO baselines and representative two-stage and transformer detectors, together with a component ablation. The retained result improves AP from 59.3% for YOLOv8n to 66.3% for TFRA-YOLO at a modest 0.38 M-parameter and 0.8 G-FLOP increase.

Unlike a fully autonomous control claim, the proposed workflow deliberately retains on-site operator confirmation. The cloud digital-twin platform provides live equipment-state visualization, defect analytics, and authorized remote supervisory control. The latency-critical defect-to-stop path remains local and wired; it does not depend on a cloud return path. The system is not claimed to perform unattended online learning or safety-certified machine control.

II. Closed-Loop Inspection Workflow

A. Production Workflow

Fig. 1 illustrates the intended operating sequence. An operator starts the circular knitting machine. A proximity switch reports the machine rotation state to the edge side and enables image acquisition and inference. The edge unit analyzes captured fabric images locally. When its decision logic identifies a candidate defect, it sends a stop request through the machine-side control interface and records the associated image and prediction. The candidate is shown locally to the operator for secondary confirmation. After the operator handles the event and restarts the machine, the production cycle resumes. Device states, detection records, and local confirmation outcomes are synchronized to a cloud digital-twin platform, where they support real-time status visualization, defect-category statistics, and process-improvement analysis. Authorized users can also issue remote supervisory control actions through the platform; these actions are distinct from the local wired defect-to-stop path.

B. On-Machine Deployment Configuration

The deployed edge terminal uses a Rockchip RK3566 processor running Android 13. Images are acquired with a SmartSens 1.3-megapixel industrial high-definition global-shutter camera at 1280×1024 resolution and a nominal 30 frames/s. A 30-level adjustable supplementary light is used to stabilize the local imaging condition. The camera is installed 10 cm from the fabric, providing an approximately 16×14 cm field of view. On the deployed terminal, the reported inference time is 45 ms per processed image (approximately 22.2 processed frames/s). This rate is sufficient for the maximum stated machine condition: a 38-inch circular knitting machine at 30 revolutions/min has a circumference of approximately 3.03 m and a fabric-surface speed of approximately 1.52 m/s. Consequently, the surface advances approximately 6.8 cm during one 45-ms inference interval, which remains below the 14-cm shorter side of the field of view and yields at least 7.2 cm nominal overlap between adjacent processed views. The edge terminal is connected to the circular knitting machine control system by a direct wired interface; the measured time from a defect decision to transmission of the stop-control signal is below 10 ms. This value characterizes the electrical control-command path, not the physical time required for the machine to come to a full stop. A 4G wireless communication module connects the edge terminal with the cloud IoT platform for non-latency-critical status and defect-record synchronization. The system is designed to support continuous 24/7 inspection. Table I summarizes the configuration.

TABLE I
On-machine deployment configuration.

Item	Configuration or measured value
Edge terminal	Rockchip RK3566; Android 13
Camera	SmartSens 1.3-megapixel industrial HD global-shutter camera
Supplementary light	30-level adjustable illumination
Acquisition	1280×1024; nominal 30 frames/s
Installation geometry	10 cm camera-to-fabric distance; approximately 16×14 cm field of view
Inference time	45 ms per processed image (approximately 22.2 processed frames/s)
Maximum-speed coverage	38-inch diameter and 30 revolutions/min: 1.52 m/s surface speed; 6.8 cm advance per processed view
Control interface	Direct wired connection to the circular knitting machine controller
Decision-to-command latency	< 10 ms from defect decision to stop-control signal transmission
Cloud communication	4G wireless module; MQTT-based synchronization
Cloud platform	Digital twin: live state, defect analytics, and authorized remote control
Operating mode	Designed for continuous 24/7 inspection

C. Platform-Aware Baseline Selection and NPU Deployment

The choice of YOLOv8n as the baseline is driven by deployment compatibility rather than by version chronology alone. The RKNN Model Zoo provides tested RK3566 NPU examples and single-core INT8 throughput figures for several YOLO families at 640×640 input [21]. In the current official benchmark, YOLOv8n reaches 34.0 frames/s, exceeding the listed newer YOLOv10n (20.7 frames/s) and YOLO11n (20.6 frames/s). The legacy YOLOv5n and YOLOv6n entries are faster (39.7 and 48.8 frames/s, respectively), which directly demonstrates that a newer YOLO version is not automatically a better NPU choice. We therefore select YOLOv8n as a high-throughput, officially supported, and structurally extensible baseline for textile-specific modification, rather than claiming that it is the absolute fastest model in the official benchmark. The benchmark values are used only for platform-aware model selection: they do not substitute for measuring the final customized detector on the deployed application.

The training and deployment models use different representations. Training produces a PyTorch checkpoint, which is

exported at 640×640 through ONNX and then converted to an RKNN model for the RK3566 NPU. We use the official RKNN-Toolkit2 conversion workflow, which targets Rockchip NPUs and supports model conversion, quantization, inference, and performance evaluation [22]. Integer-only quantization is a well-established route to reduce inference cost on constrained hardware [23]. In the deployment path, post-training INT8 quantization is applied during ONNX-to-RKNN conversion with representative training images used for calibration; the resulting RKNN artifact is loaded by the Android edge application for NPU execution. This workflow is summarized as *PyTorch checkpoint* $\rightarrow$ *ONNX* $\rightarrow$ *INT8 RKNN* $\rightarrow$ *RK3566 NPU*. The reported 45-ms edge inference time in Table I corresponds to the deployed application, not PyTorch inference on the training workstation.

Fig. 2 provides the actual on-machine installation. The wide view shows the circular knitting machine on the production floor. The upper inset highlights the camera directed at the knitting region, and the lower inset shows the edge-side touch display during inspection. Together with the workflow in Fig. 1, this installation evidence distinguishes the work from an offline-only image experiment.

Fig. 3 shows the corresponding cloud IoT platform. The platform provides a portal view, defect-count trend statistics, device status and remote configuration, and a review page for uploaded defect records. These cloud functions support supervisory operation and process analysis; they are separated from the latency-critical local wired stop path.

The design follows the edge-intelligence principle that time-sensitive perception and action remain near the physical process, while selected evidence can be retained centrally. Here, the control consequence is more direct: the event is escalated to a worker, not accepted as a final autonomous verdict. This provides a pragmatic route for handling rare defects while retaining the records needed to identify recurring defect patterns and process anomalies.

D. Operational Boundaries

The workflow separates three decisions: (i) whether machine rotation enables inspection, (ii) whether the detector produces a candidate defect, and (iii) whether an operator confirms and disposes of the event. The reported recognition experiments validate only the second decision on a held-out image split. The deployed configuration establishes geometric image coverage at the stated maximum machine speed, the camera, edge terminal, wired control interface, and decision-to-command latency, but does not yet quantify physical machine-stop time, false-stop rate on defect-free fabric, frame-delivery or drop statistics during continuous operation, or operator-confirmation rate. These measurements should be reported from a controlled site trial before submission.

III. TFRA-YOLO

A. Overview

TFRA-YOLO is built from the YOLOv8n detector [24]. As shown in Fig. 4, the backbone retains the original resolution hierarchy, but replaces the C2f output refinement at four stages with C2f–WWTE. In the head, the two top-down nearest-neighbor upsampling operations are replaced with LTFR. The standard concatenation, C2f fusion, bottom-up path, and three-scale detection head are retained. The detection scales are P_3, P_4, and P_5, corresponding to 80×80, 40×40, and 20×20 feature maps for a 640×640 input.

B. Warp–Weft Texture Encoding

The visual cue of a defect in plain-knit fabric is usually not an isolated object boundary. Instead, it is a local violation of the repetitive loop texture: a horizontal defect interrupts continuity across the width direction, a vertical defect interrupts continuity along the height direction, and a hole produces a compact local discontinuity. A standard C2f output preserves multi-path features, but its local convolutions do not explicitly distinguish these three forms of texture interruption. The term "warp–weft" is retained as the internal WWTE name because it denotes two orthogonal texture directions. In this circular-knit application, it does *not* imply that the fabric is woven; operationally, the branches are interpreted as horizontal and vertical texture encoders.

WWTE is inserted *after* the selected C2f outputs rather than replacing the C2f gradient-flow structure. Thus, the original pretrained representation is retained, while WWTE learns a residual texture correction at multiple backbone depths. For a C2f output $X \in \mathbb{R}^{C \times H \times W}$, the module applies depthwise horizontal, vertical, and local branches and then fuses them:

$$F(X) = \phi\left([\mathrm{DW}_{1\times7}(X), \mathrm{DW}_{7\times1}(X), \mathrm{DW}_{3\times3}(X)]\right), \quad (1)$$

where $[\cdot]$ denotes channel concatenation and ϕ is a 1×1 convolution with normalization and activation. The 1×7 branch aggregates evidence along a horizontal trace without first mixing channels, whereas the 7×1 branch performs the analogous vertical aggregation. The 3×3 branch retains compact local evidence required for holes and short texture breaks. Depthwise filtering keeps the long directional receptive fields inexpensive; the following pointwise fusion learns how much horizontal, vertical, and local evidence should be retained at each feature channel. Because input and output channels match in the present implementation, the module uses residual fusion:

$$Y = X + F(X). \quad (2)$$

Equation (2) is important for an edge-deployed adaptation: the C2f feature X remains directly available if a directional response is unhelpful, while $F(X)$ adds only the learned, task-specific texture cue. The design is related to, but not equivalent to, asymmetric convolution blocks [17]: it uses depthwise directional paths, three-way concatenation, and a learned pointwise fusion within the existing C2f structure. Its intended effect is therefore selective sensitivity to elongated horizontal and vertical discontinuities without sacrificing the compact context used for holes.

C. Local Texture-Guided Feature Reassembly

The features enriched by WWTE must next be passed from a coarse semantic level to a finer detection level. Fixed nearest-neighbor upsampling simply copies a source value to

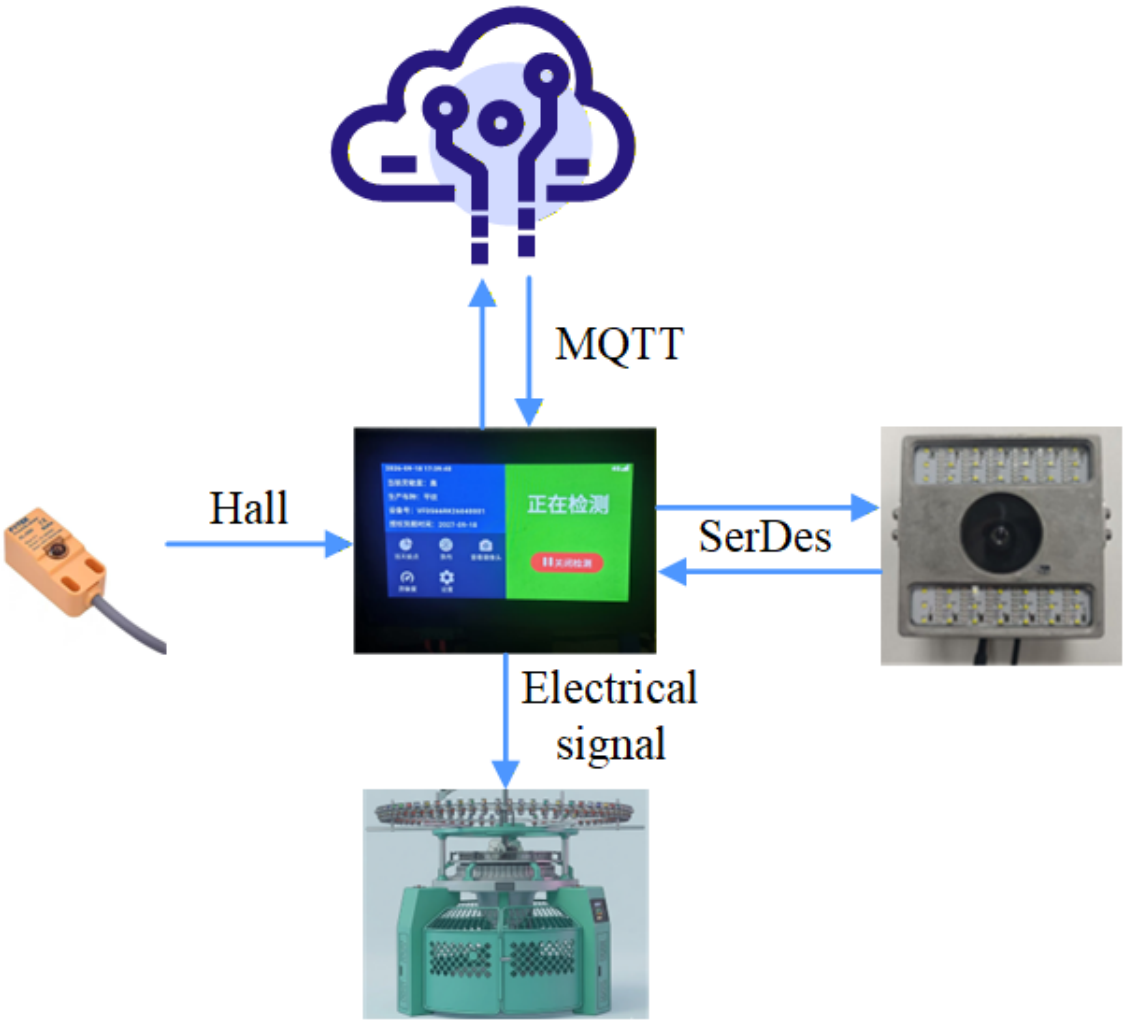


Fig. 1. System overview of the deployed circular-knitting inspection setup. A Hall sensor reports the machine rotation state to the edge terminal; the industrial camera sends images through a SerDes link; the edge terminal issues the wired electrical stop-control signal to the machine; and the cloud platform exchanges status and inspection records through MQTT over the 4G wireless link.

a 2×2 region; it has no mechanism to decide whether the copied response corresponds to regular fabric texture, a thin directional break, or a defect boundary. This limitation is most consequential in the two top-down transitions, where coarse semantic features must be aligned with the high-resolution texture evidence used by the P_4 and P_3 detectors. LTFR therefore replaces only these two fixed upsampling operations; the original concatenation, C2f fusion, bottom-up path, and detection head remain unchanged.

LTFR instantiates the content-aware reassembly principle of CARAFE [16] in this restricted pyramid role. Given $X \in \mathbb{R}^{C\times H\times W}$, a 1×1 compressor first reduces the kernel-prediction feature to $C_r = \max(C/4, 16)$ channels. A 3×3 encoder then produces s^2k^2 kernel logits, where the scale factor is $s = 2$ and the reassembly kernel size is $k = 5$. Pixel shuffle expands these logits onto the upsampled grid, and softmax yields a distinct normalized 5×5 kernel $K_{i,j}$ at each output location:

$$K_{i,j} = \mathrm{softmax}\left(\mathrm{PS}\left(E_{3\times3}(E_{1\times1}(X))\right)_{i,j}\right). \tag{3}$$

Here, $E_{1\times1}$ and $E_{3\times3}$ denote the compression and kernel-encoding layers, respectively, and PS denotes pixel shuffle. The 25 weights of each $K_{i,j}$ sum to one. For each output position, a 5×5 neighborhood $\mathcal{N}_{i,j}(X)$ is unfolded from X, spatially aligned with the enlarged grid, and reassembled as

$$Y_{i,j} = \sum_{u=1}^{25} K_{i,j,u}\mathcal{N}_{i,j,u}(X). \tag{4}$$

Unlike a spatially shared interpolation kernel, Eq. (4) allows a location near a thin defect or a texture boundary to select a different local mixture from a location within regular fabric. The channel-compression stage bounds the overhead of predicting these kernels, while the 5×5 neighborhood provides more local support than a copied source pixel. LTFR is not claimed as a new generic reassembly primitive; its contribution here is the lightweight placement of input-dependent reassembly at the two texture-critical top-down transitions of YOLOv8n.

WWTE and LTFR have complementary roles. WWTE first creates direction-sensitive cues in the backbone; LTFR then propagates and aligns those cues during top-down reconstruction instead of indiscriminately replicating them. The

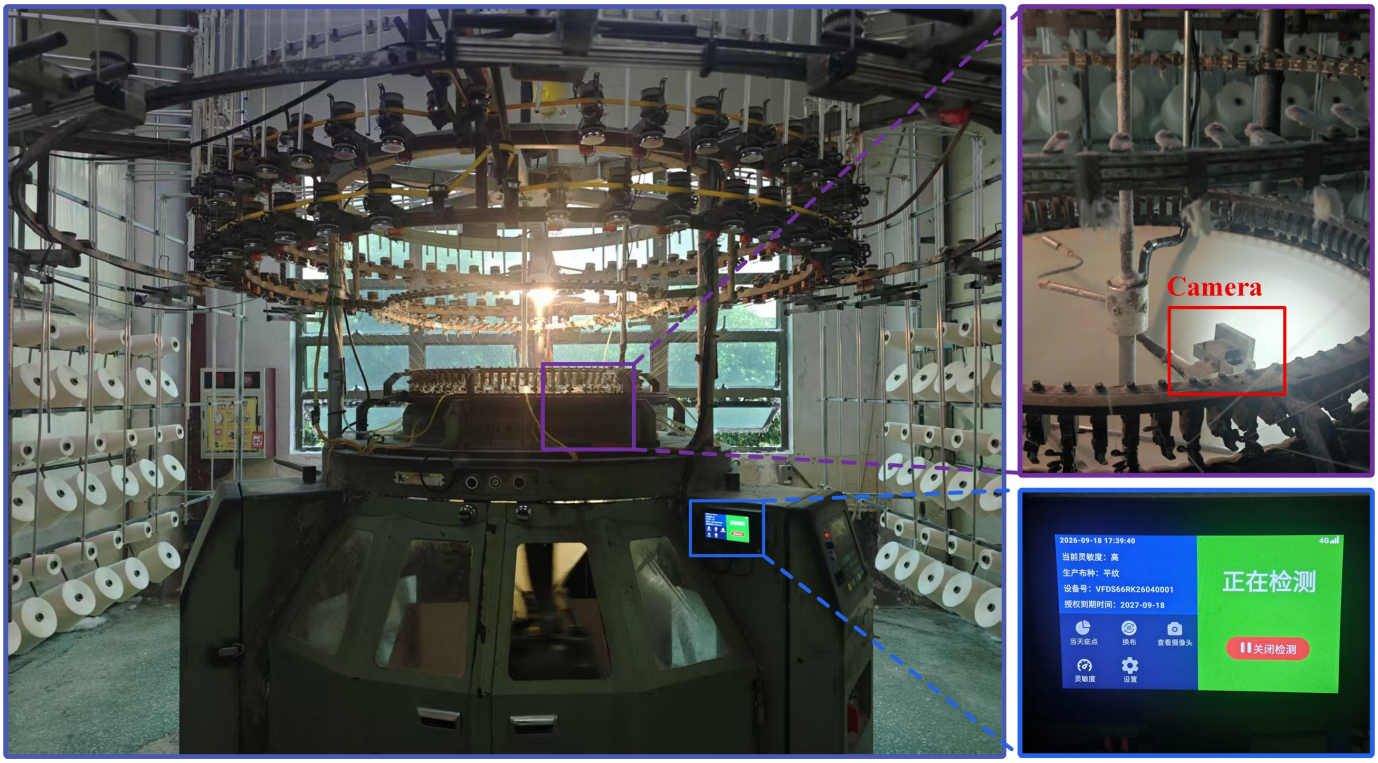


Fig. 2. Actual production-line installation on the circular knitting machine. The upper inset highlights the camera location; the lower inset shows the edge-side touch display while inspection is active. Colored callouts and insets are retained from the installation record.

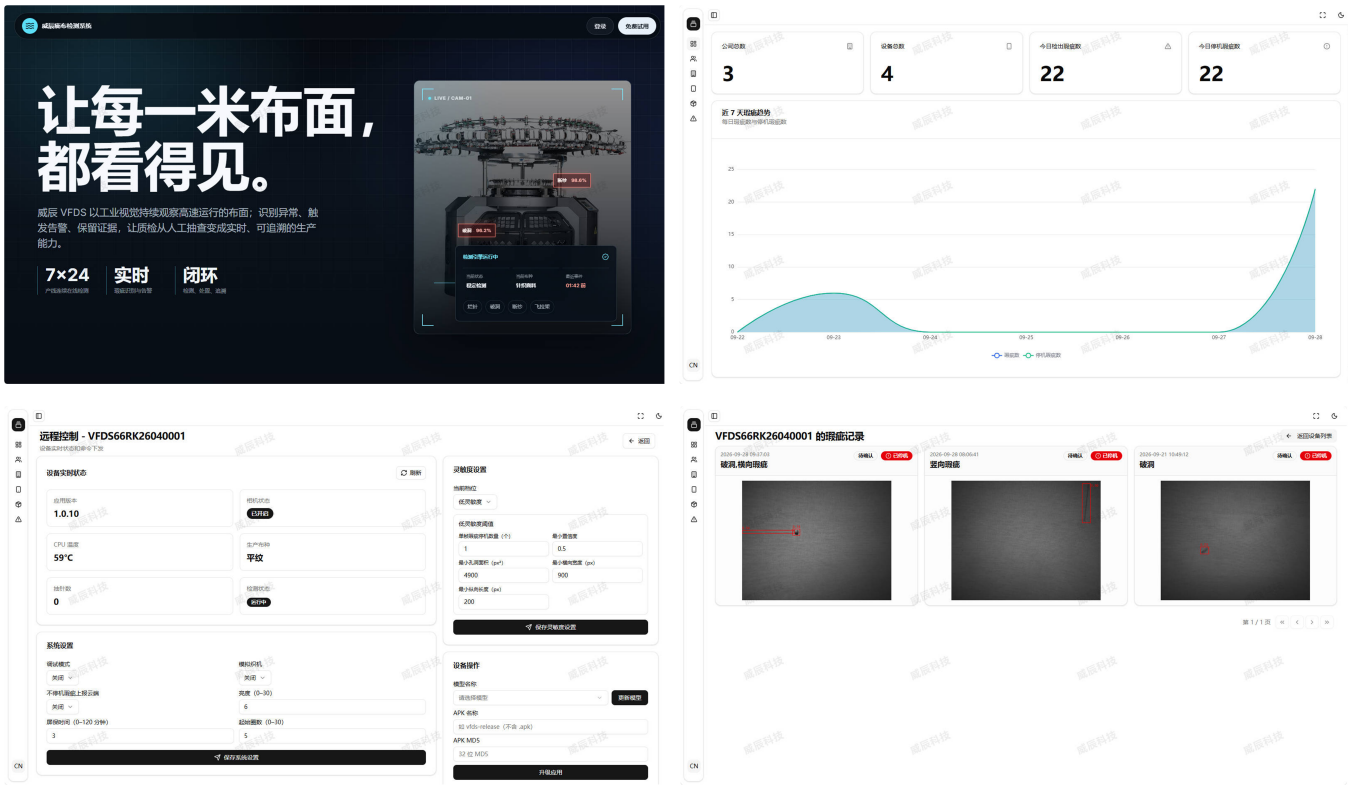


Fig. 3. Cloud IoT platform for the deployed inspection system. The four panels show the platform portal, defect-count trend dashboard, device-state and remote-configuration interface, and uploaded defect-record review interface, respectively. Cloud communication supports status visualization, statistics, and authorized supervision, whereas the defect-to-stop control path remains local and wired.

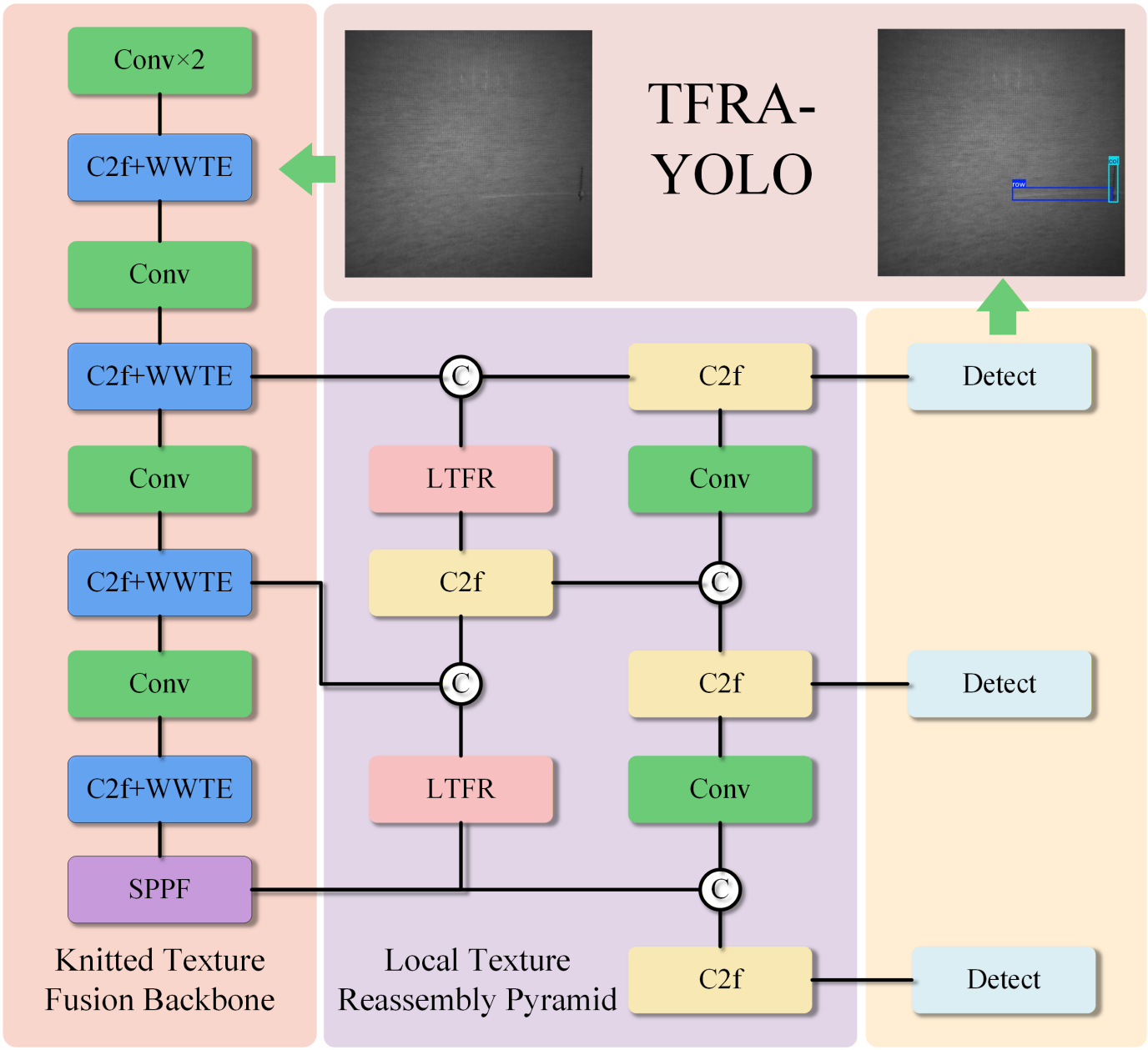


Fig. 4. Architecture of TFRA-YOLO. The Knitted Texture Fusion Backbone uses C2f–WWTE blocks to encode horizontal, vertical, and local texture cues. The Local Texture Reassembly Pyramid replaces the two top-down fixed upsampling operations with LTFR, while the bottom-up path and three-scale detection head follow YOLOv8n.

component ablation in Table V tests this design: WWTE produces the larger isolated gain, LTFR still improves the baseline on its own, and their combination yields the best result. Figs. 5 and 6 detail the two modules.

IV. Experimental Protocol

A. Dataset

The data were collected from a real textile production line. The three object categories are *horizontal defect*, *vertical defect*, and *hole*. Table II summarizes the provided image-level split and instance counts. The test set contains 81 images with 91 labeled defects. This small, imbalanced test set makes every retained result useful for engineering comparison but is insufficient by itself to support a broad generalization claim. A future submission should state whether the split is grouped by production roll, machine, material, or time interval, and should add an independent production-period test set if available.

TABLE II
Dataset split supplied for the current draft.

Split	Images	Instances	Horizontal	Vertical	Hole
Train	282	329	174	75	80
Validation	40	48	24	13	11
Test	81	91	60	12	19

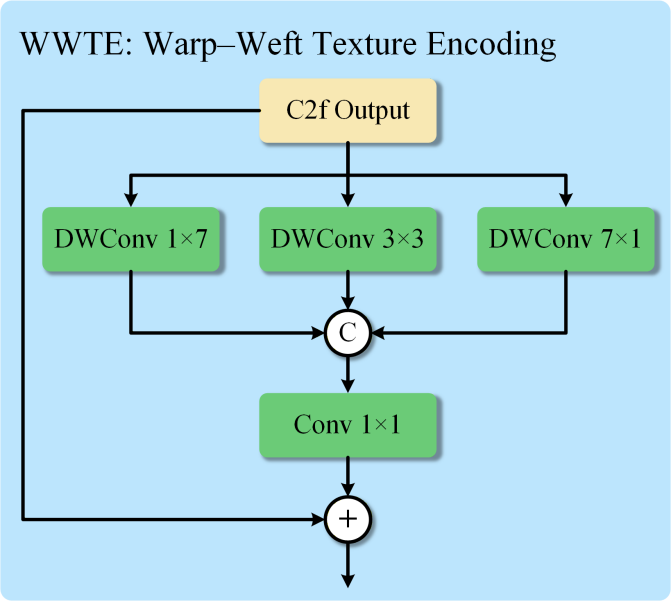


Fig. 5. WWTE structure. A C2f feature is processed by horizontal, vertical, and local depthwise branches, followed by channel concatenation, 1×1 fusion, and residual addition.

B. Training and Evaluation

Table III summarizes the implementation protocol used for the Ultralytics experiments. TFRA-YOLO and its direct YOLOv8n baseline are initialized from `yolov8n.pt`; the remaining comparison models use the available official pretrained initialization in their respective repositories. Evaluation is conducted on the held-out test split with a maximum of 16 detections per image.

C. Evaluation Metrics

We use the COCO detection protocol. For a predicted box B_p and a ground-truth box B_g, intersection over union (IoU) is defined as $\mathrm{IoU}(B_p, B_g) = |B_p \cap B_g|/|B_p \cup B_g|$. At an IoU threshold t, a prediction is counted as a true positive only when it has the correct class and is matched one-to-one to an as-yet unmatched ground-truth instance; all remaining predictions are false positives and unmatched ground-truth instances are false negatives. For each class c, $\mathrm{AP}_c(t)$ is the area under its interpolated precision–recall curve. With $C = 3$ classes, the reported AP and AP50 are

$$\mathrm{AP} = \frac{1}{10C}\sum_{c=1}^{C}\sum_{t\in\mathcal{T}}\mathrm{AP}_c(t),$$
$$\mathrm{AP50} = \frac{1}{C}\sum_{c=1}^{C}\mathrm{AP}_c(0.50), \tag{5}$$

where $\mathcal{T} = \{0.50, 0.55, \ldots, 0.95\}$. Thus, AP measures localization and classification quality across increasingly strict overlap thresholds, whereas AP50 emphasizes detection at IoU $= 0.50$. Both are written as percentages in the tables. We additionally report trainable parameters and FLOPs at $[1, 3, 640, 640]$ to characterize model scale and computational cost; these are not substitutes for measured edge-device latency.

For each model, the retained best-validation checkpoint is evaluated once on the same held-out test split. FLOPs and parameter counts are reported at $[1, 3, 640, 640]$. The values are suitable for a coarse efficiency comparison, but cross-framework FLOP accounting can omit unsupported operators; no cross-framework latency claim is made. DETR, Deformable DETR, Faster R-CNN, and RT-DETRv2 follow their respective repositories and pretrained backbones [25]–[28].

V. Results and Discussion

A. Comparison With Detectors

Table IV compares TFRA-YOLO with the retained baselines. TFRA-YOLO achieves 66.3% AP and 96.9% AP50. Relative to its direct YOLOv8n baseline, AP improves by 7.0 points and AP50 by 2.8 points for 0.38 M additional parameters and 0.8 G additional FLOPs. Among the compact YOLO variants evaluated here, TFRA-YOLO has the highest AP. RT-DETR-R18 obtains 67.7% AP, 1.4 points higher than TFRA-YOLO, but requires 20.09 M parameters and 60.72 G FLOPs. This is a useful trade-off rather than a claim that TFRA-YOLO dominates every detector.

Fig. 7 visualizes the parameter- and operation-accuracy trade-off without shifting any data coordinates. The position of TFRA-YOLO in the low-parameter region is consistent with the tabulated result. Because the horizontal axes are logarithmic, the visual is intended to show scale rather than imply a linear performance-efficiency relationship.

B. Ablation

Table V isolates the two modifications within the same YOLOv8n family. WWTE contributes the larger isolated improvement (+4.8 AP), while LTFR contributes +1.6 AP. Combining both gives 66.3% AP, which is 0.6 points above the arithmetic sum of the isolated gains. This indicates that the texture representation and adaptive reassembly are complementary: WWTE supplies direction-specific texture cues, whereas LTFR transfers them through the top-down path.

C. Per-Class and Qualitative Results

The TFRA-YOLO checkpoint obtains 61.7%, 73.8%, and 63.4% AP for horizontal defects, vertical defects, and holes, respectively. The notably high vertical-defect AP should be interpreted with care because the test category has only 12 instances. Fig. 8 presents seven selected images and the available standardized renderings. The visual comparison is illustrative rather than a replacement for the full quantitative test set. It is particularly useful for examining thin vertical defects, long horizontal defects, and the hole class against repetitive background texture.

Fig. 9 further compares the activation visualizations produced by the official Ultralytics visualization routine for the YOLOv8n baseline and TFRA-YOLO. Each row corresponds to the same test image, with the original image, baseline visualization, and TFRA-YOLO visualization arranged from

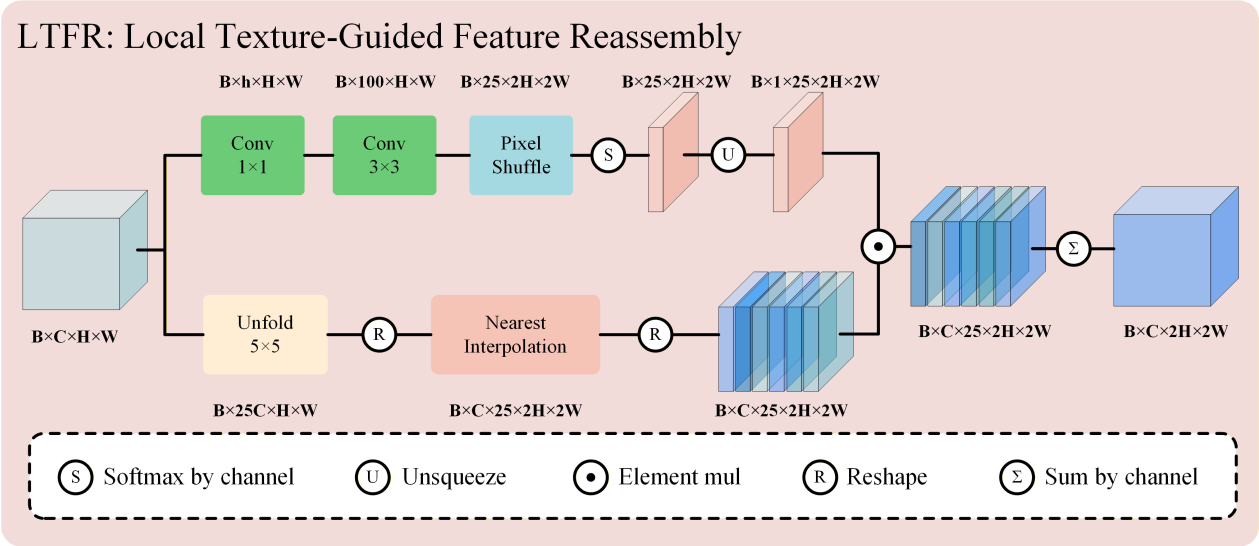


Fig. 6. LTFR structure. The upper path predicts location-specific 5×5 reassembly kernels, and the lower path unfolds and aligns local feature patches. Their element-wise product is summed along the kernel dimension to produce a $2\times$ upsampled feature.

TABLE III
IMPLEMENTATION PROTOCOL AND TRAINING HYPERPARAMETERS.

Item	Specification
Training hardware	NVIDIA GeForce RTX 3090 GPU (24 GB)
Software	Ubuntu 24.04, Python 3.10.20, PyTorch 2.4.1+cu124, and Ultralytics 8.4.140
Model initialization	TFRA-YOLO and YOLOv8n: `yolov8n.pt`; other baselines: the available official pretrained checkpoint in the corresponding repository
Input resolution	640×640
Batch size / data workers	16 / 8
Maximum epochs / early stopping patience	300 / 80 epochs
Optimizer	Ultralytics automatic optimizer selection
Data augmentation	Standard Ultralytics geometric and photometric augmentations; no augmentation hyperparameter was manually overridden
Checkpoint retained for testing	Framework-selected best validation checkpoint
Test protocol	Held-out test split, maximum 16 detections per image

TABLE IV
TEST-SET COMPARISON ON 81 IMAGES AND 91 INSTANCES. AP IS COCO AP@[.50:.95]. ALL VALUES ARE PERCENTAGES EXCEPT PARAMETERS AND FLOPS.

Model	Params (M)	FLOPs (G)	AP50 (%)	AP (%)
YOLOv5n[†]	2.50	7.10	93.1	58.4
YOLOv8n	3.01	8.10	94.1	59.3
YOLO11n	2.58	6.40	95.0	59.6
YOLO12n	2.56	7.30	94.3	60.9
YOLO26n	2.38	5.30	96.5	61.4
RT-DETR-R18	20.09	60.72	95.2	**67.7**
DETR-R50	41.28	77.25	96.4	53.6
Deformable DETR-R50	39.82	158.66	96.8	64.3
Faster R-CNN-R50	41.13	181.45	96.1	64.7
TFRA-YOLO	3.39	8.90	**96.9**	66.3

[†]The retained YOLOv5n checkpoint is the legacy YOLOv5n architecture, not the Ultralytics YOLOv5u variant.

left to right. These maps are used only as qualitative evidence of where the detectors respond; the AP results in Tables IV and V remain the basis for the quantitative comparison.

D. Limitations and Submission-Ready Validation

The present evidence supports a promising lightweight detector on a real production-line image set, but it does not yet establish production reliability. First, the test split is small and category-imbalanced. Second, FLOPs are not a substitute

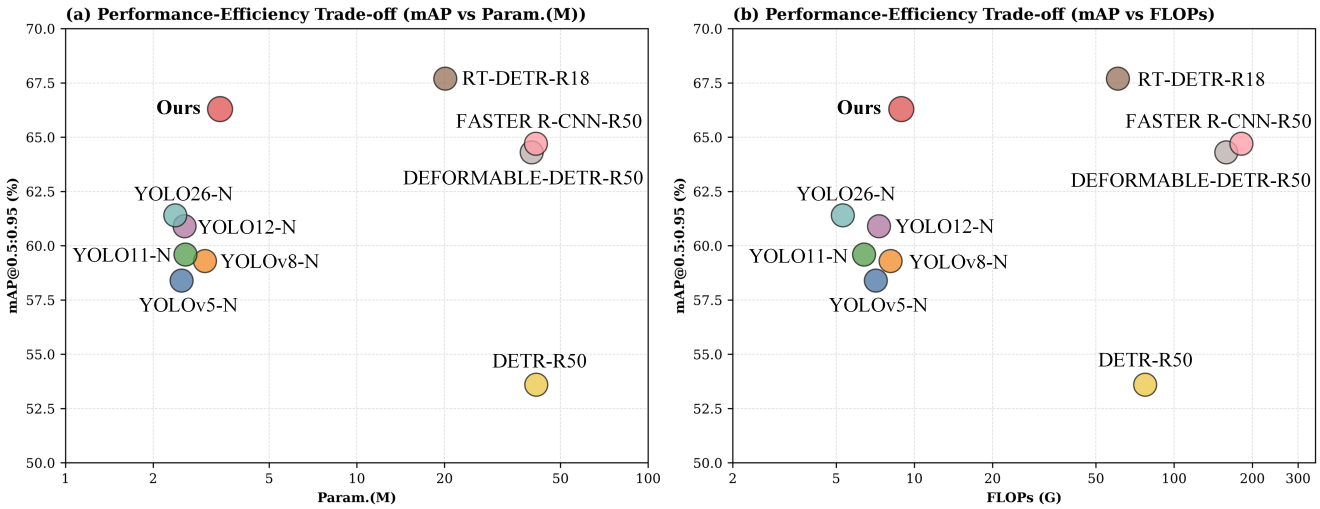


Fig. 7. Accuracy-efficiency trade-off of the retained test results. Both horizontal axes use a logarithmic scale. Marker coordinates are the exact values in Table IV; the plot does not contain manual coordinate compression.

TABLE V
ABLATION ON THE COMMON HELD-OUT TEST SPLIT USING THE SAME YOLOV8N TRAINING RECIPE.

Variant	WWTE	LTFR	Params	FLOPs	AP50	AP
YOLOv8n	–	–	3.01	8.1	94.1	59.3
+ WWTE	✓	–	3.28	8.8	94.2	64.1
+ LTFR	–	✓	3.11	8.2	94.3	60.9
TFRA-YOLO	✓	✓	3.39	8.9	96.9	66.3

for end-to-end edge latency, especially when image capture, preprocessing, postprocessing, control communication, and physical stopping are part of the operational loop. Finally, the operator's confirmation records are intended to improve defect statistics and process analysis; their actual acceptance rate and operational value have not yet been measured. These points define a concrete validation plan, rather than weaknesses that should be concealed.

VI. CONCLUSION

This work presents a closed-loop framework for defect inspection on a circular knitting machine and TFRA-YOLO, a compact direction- and content-adaptive detector. The workflow couples rotation-aware edge inspection, local wired stop signaling, on-site human confirmation, and a cloud digital twin for live status visualization, defect statistics, process improvement, and authorized remote supervisory control. The detector combines WWTE for horizontal, vertical, and local knitted-texture encoding with LTFR for adaptive feature-pyramid reconstruction. On the retained test split, TFRA-YOLO improves YOLOv8n from 59.3% to 66.3% AP at 3.39 M parameters and 8.9 G FLOPs. The next step is not merely more offline accuracy: it is a disciplined site evaluation of sensing-to-stop latency, false-stop behavior, confirmation outcomes, process-analysis value, and grouped generalization across production conditions.

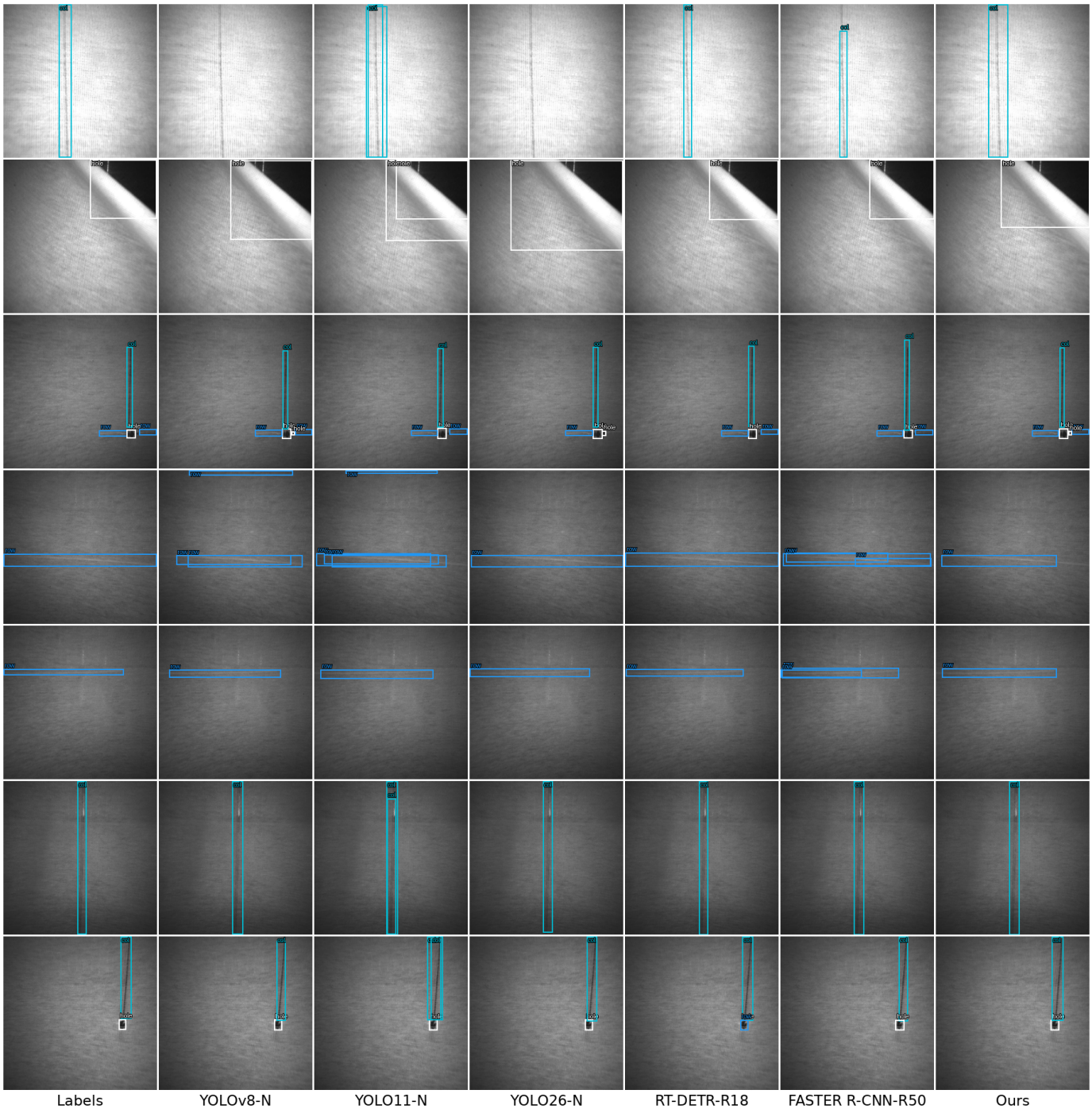


Fig. 8. Qualitative comparison on selected test images. Columns are labels, YOLOv8n, YOLO11n, YOLO26n, RT-DETR-R18, Faster R-CNN-R50, and TFRA-YOLO. The figure uses a common renderer for visual consistency; it should be updated after finalizing the exact sample-selection protocol and display confidence threshold.

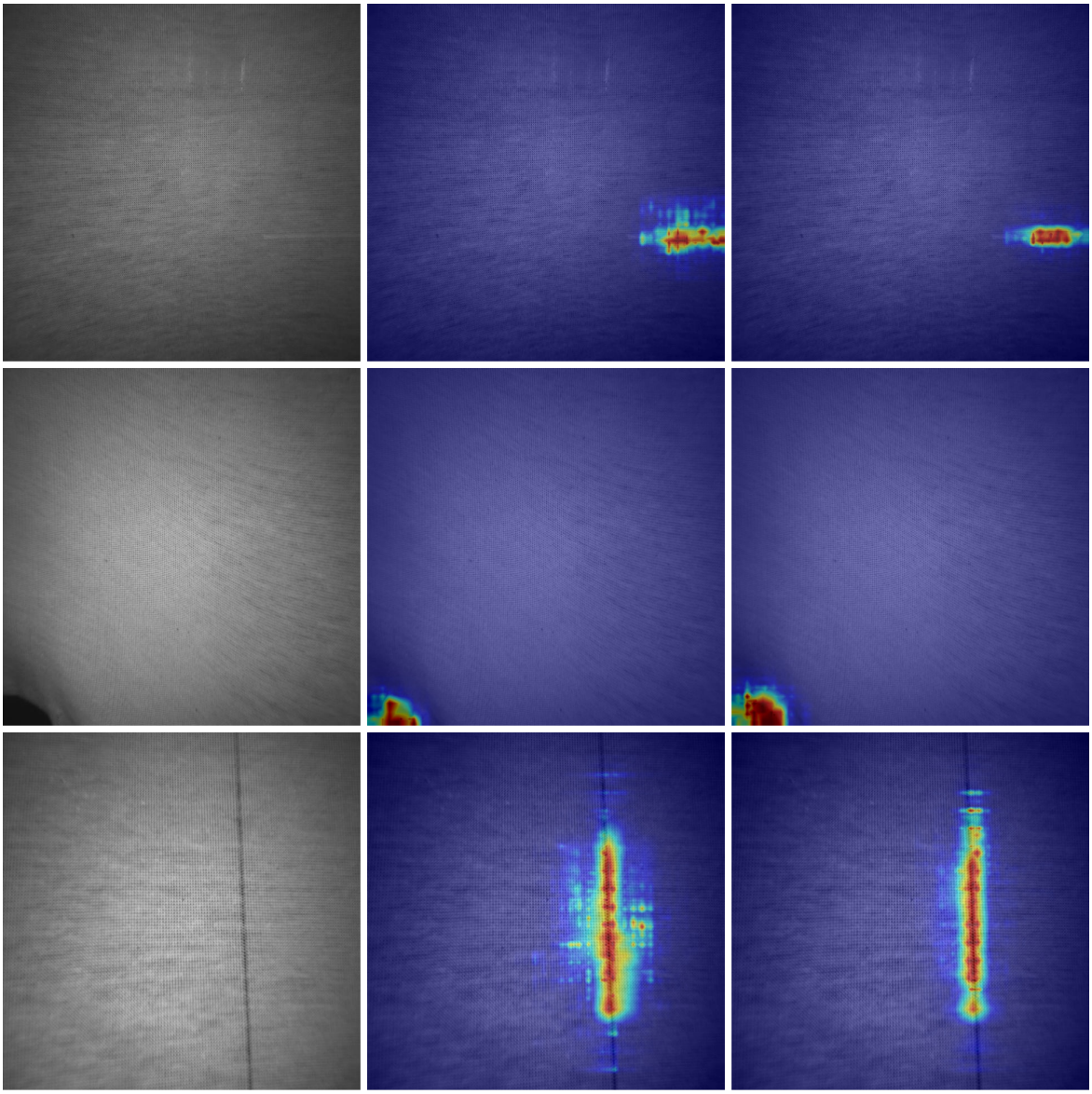


Fig. 9. Qualitative activation visualization on three selected test images. The left, middle, and right columns show the original images, YOLOv8n visualizations, and TFRA-YOLO visualizations, respectively. Each visualization is generated with the same official Ultralytics routine.

REFERENCES

[1] R. G. Saeidi, M. Latifi, S. S. Najar, and A. G. Saeidi, "Computer vision-aided fabric inspection system for on-circular knitting machine," *Textile Research Journal*, vol. 75, no. 6, pp. 492–502, 2005.

[2] Y.-Q. Ni, P.-K. Huang, C.-H. Yang, C.-C. Chang, W.-J. Wang, and D. Liang, "Real-time defect detection for fast-moving fabrics on circular knitting machine under various illumination conditions," *IEEE Access*, vol. 13, pp. 139 890–139 903, 2025.

[3] A. Kumar, "Computer-vision-based fabric defect detection: A survey," *IEEE Trans. Ind. Electron.*, vol. 55, no. 1, pp. 348–363, 2008.

[4] H. Y. T. Ngan, G. K. H. Pang, and N. H. C. Yung, "Automated fabric defect detection—a review," *Image Vis. Comput.*, vol. 29, no. 7, pp. 442–458, 2011.

[5] C. Li, J. Li, Y. Li, L. He, X. Fu, and J. Chen, "Fabric defect detection in textile manufacturing: A survey of the state of the art," *Secur. Commun. Netw.*, vol. 2021, p. 9948808, 2021.

[6] Y. Kahraman and A. Durmuşoğlu, "Deep learning-based fabric defect detection: A review," *Text. Res. J.*, vol. 93, no. 5–6, pp. 1485–1503, 2023.

[7] J.-F. Jing, H. Ma, and H.-H. Zhang, "Automatic fabric defect detection using a deep convolutional neural network," *Color. Technol.*, vol. 135, no. 3, pp. 213–223, 2019.

[8] Z. Liu, Z. Huo, C. Li, Y. Dong, and B. Li, "DLSE-Net: A robust weakly supervised network for fabric defect detection," *Displays*, vol. 68, p. 102008, 2021.

[9] X. Kang and J. Li, "AYOLOv7-tiny: Towards efficient defect detection in solid color circular weft fabric," *Text. Res. J.*, vol. 94, pp. 225–245, 2024.

[10] H. Yang, J. Jing, Z. Wang *et al.*, "YOLOV4-TinyS: A new convolutional neural architecture for real-time detection of fabric defects in edge devices," *Text. Res. J.*, vol. 94, pp. 49–68, 2024.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.

[12] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[13] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 7464–7475.

[14] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2117–2125.

[15] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path aggregation network for instance segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8759–8768.

[16] J. Wang, K. Chen, R. Xu, Z. Liu, C. C. Loy, and D. Lin, "Carafe: Content-aware reassembly of features," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 3007–3016.

[17] X. Ding, Y. Guo, G. Ding, and J. Han, "Acnet: Strengthening the kernel skeletons for powerful cnn via asymmetric convolution blocks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1911–1920.

[18] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.

[19] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019.

[20] W. Kritzinger, G. Karner, C. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.

[21] Rockchip, "Rknn model zoo," [Online]. Available: https://github.com/airockchip/rknn_model_zoo, 2026, accessed: 2026-09-27.

[22] ——, "Rknn toolkit2," [Online]. Available: https://github.com/airockchip/rknn-toolkit2, 2026, accessed: 2026-09-27.

[23] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713.

[24] Ultralytics, "Ultralytics yolo," [Online]. Available: https://github.com/ultralytics/ultralytics, 2023, accessed: 2026-09-24.

[25] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 213–229.

[26] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *Proc. Int. Conf. Learn. Representations*, 2021.

[27] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Adv. Neural Inf. Process. Syst.*, vol. 28, 2015.

[28] W. Lv, Y. Xu, Y. Yang, G. Li, J. Wang, L. Chen *et al.*, "Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer," *arXiv preprint arXiv:2407.17140*, 2024.